ESTRUCTURA DE DATOS

PROYECTOS 2025-2

Docente: Dr. Eric Gustavo Coronel Castillo

PROPÓSITO 2

ALCANCE TÉCNICO 2

ENTREGABLES 2

GLOSARIO DE TÉRMINOS 3

PROYECTO 1 Historial de navegación con lista doblemente enlazada 6

PROYECTO 2 Lista de reproducción de música con lista circular 8

PROYECTO 3 Carrito de compras en línea con lista simplemente enlazada 10

PROYECTO 4 Gestor de tareas pendientes con lista doblemente enlazada 12

PROYECTO 5 Sistema de impresión con cola de trabajos 14

PROYECTO 6 Atención en banco con preferencia para adultos mayores 16

PROYECTO 7 Atención en municipalidad con múltiples servicios y cola de apoyo 18

PROYECTO 8 Centro de llamadas multicanal con lista circular de agentes 20

PROYECTO 9 Estacionamiento en una sola vía con carril auxiliar 22

PROYECTO 10 Verificador de expresiones con paréntesis y corchetes balanceados 24

PROYECTO 11 Control de seguridad en aeropuerto: bandejas y filas 26

PROYECTO 12 Biblioteca universitaria: índice de libros por ABB 28

PROYECTO 13 Sistema de decisiones médicas con árbol binario de expresiones 30

PROYECTO 14 Calculadora de expresiones aritméticas con árbol de expresión 32

PROPÓSITO

Desarrollar soluciones a problemas reales aplicando listas enlazadas (simples, dobles y circulares), pilas, colas y árboles binarios de búsqueda (ABB), con la capacidad de explicar por qué se elige cada estructura y cómo impacta en el desempeño de la solución.

ALCANCE TÉCNICO

La implementación se realiza en C# (.NET 6 o superior). No se permite usar colecciones de alto nivel para sustituir las estructuras requeridas. Por ejemplo: no usar List<T> o LinkedList<T> como almacenamiento principal. Las estructuras deben ser de autoría propia. Solo se permite usar colecciones de la biblioteca estándar para utilidades externas, como la carga de datos desde archivos CSV.

ENTREGABLES

1. Código fuente en C# con estructuras implementadas por el equipo (no sustituir por colecciones de alto nivel).
2. Informe técnico: descripción del problema, diseño y justificación de la EDD elegida, decisiones clave, análisis de desempeño (empírico y/o Big-O), pruebas realizadas y conclusiones.
3. Video explicativo (mínimo 15 min): presentación del problema, diagrama de la EDD, decisiones de diseño, demostración de ejecución, resultados y conclusiones. Con cámara activa y participación de todos los integrantes.
4. Demostración en vivo (15 min): ejecución guiada con un conjunto de datos representativo, casos borde y defensa técnica de las decisiones.
5. Guía de ejecución, conjuntos de datos (datasets) y material de presentación (diapositivas o guion).

GLOSARIO DE TÉRMINOS

Este glosario resume los términos usados en los proyectos y la rúbrica. Lenguaje claro, definiciones breves y un ejemplo de uso en el curso.

TÉRMINO	DEFINICIÓN	EJEMPLO EN EL CURSO
EDD	Estructuras de Datos. Conjunto de modelos y técnicas para organizar y manipular información (listas, pilas, colas, árboles, grafos).	Curso completo.
Nodo	Unidad básica de una lista: contiene datos y enlaces (referencias) a otros nodos.	Listas enlazadas.
Cabeza (head) / Cola (tail)	Primer y último nodo de una lista o cola, respectivamente.	Listas/colas.
Lista simplemente enlazada	Cada nodo apunta al siguiente. Recorrido hacia adelante.	Implementación de colas, historiales.
Lista doblemente enlazada	Cada nodo apunta al siguiente y al anterior. Navegación en ambos sentidos.	Listas de reproducción, pedidos en salón.
Lista circular	El último nodo enlaza con el primero. Útil para rotaciones.	Round-robin de agentes/repartidores.
Pila (stack)	LIFO: el último en entrar es el primero en salir.	Bandejas en aeropuerto; garaje en una sola vía.
Cola (queue)	FIFO: el primero en entrar es el primero en salir.	Filas de atención, tickets por canal.
Cola prioritaria	Atiende según prioridad. En empates se puede respetar el orden de llegada (estabilidad).	Triaje hospitalario.
Cola de apoyo (overflow)	Cola extra que se habilita cuando la cola principal supera un umbral (longitud o espera).	Atención municipal.
Envejecimiento (aging)	Regla que garantiza turnos a menor prioridad tras X atenciones preferentes o T minutos de espera.	Banco con preferencia a adultos mayores.
BFS (Búsqueda en Anchura)	Recorre por niveles y obtiene distancias mínimas en número de tramos en grafos no ponderados.	Rutas peatonales en campus.
DFS (Búsqueda en Profundidad)	Explora lo más profundo antes de retroceder. Útil para detección de componentes y ciclos.	Análisis de grafos.

Ordenamiento topológico	Orden de tareas/cursos en un DAG sin violar prerequisitos (algoritmo de Kahn o DFS).	Plan de estudios.
DAG	Grafo dirigido acíclico.	Prerequisitos de cursos.
Dijkstra	Caminos mínimos con pesos no negativos.	Logística urbana (rutas de reparto).
MST (Árbol de Expansión Mínima)	Subconjunto de aristas que conecta todos los nodos con costo total mínimo.	Red de sedes (Kruskal/Prim).
Union-Find (DSU)	Estructura para conjuntos disjuntos: une y consulta componentes rápidamente.	Kruskal para MST.
Grafo no ponderado	Todas las aristas tienen el mismo costo.	Rutas por número de tramos (BFS).
Grafo ponderado	Las aristas tienen costos/pesos distintos (distancia, tiempo).	Rutas con tiempos reales (Dijkstra).
Grafo disperso	Pocas aristas respecto del máximo posible; conviene lista de adyacencia.	Mapas de calles/campus.
Lista de adyacencia	Representación de un grafo con las vecindades de cada nodo.	BFS/DFS/Dijkstra.
Predecesor	Nodo desde el que se descubrió otro; permite reconstruir rutas.	Rutas mínimas.
p90 / p95 / p99	Percentiles: valor que el 90/95/99% de los casos no supera.	Reportes de tiempos de espera.
Mediana (p50)	La mitad de los casos está por debajo y la otra mitad por encima.	Resumen robusto de tiempos.
Tiempo medio (promedio)	Suma de tiempos dividido entre el número de casos.	Indicador general de desempeño.
Big-O / Notación asintótica	Forma de acotar el crecimiento del costo (O, Ω, Θ).	Análisis de operaciones (opcional).
O(1) / O(n)	Constante / lineal: costo no depende de n / crece con n.	Insertar cola vs. recorrer lista.
CI local	Pipeline de integración continua ejecutado en la máquina del equipo (build + pruebas + métricas).	Reproducibilidad de entregas.
.NET Stopwatch	Cronómetro de alta resolución para medir tiempos.	Mediciones empíricas.
BenchmarkDotNet	Framework de micro-benchmarks para .NET.	Mediciones precisas (opcional).

Dataset	Conjunto de datos para pruebas (CSV/JSON/Excel).	Simulaciones y reportes.
CSV	Archivo de valores separados por comas, usualmente con encabezado.	Datasets de llegadas/servicios.
Round-robin	Asignación cíclica y equitativa de turnos.	Agentes de call center; repartidores.
Umbral	Límite a partir del cual se activa una política.	Activar cola de apoyo.
Caso borde	Situación extrema o poco frecuente que debe manejarse correctamente.	Cola vacía; stock insuficiente.
Recorrido in-order	Visita izquierda → raíz → derecha. Produce los elementos en orden ascendente en un ABB.	Listados ordenados
Recorrido pre-order	Visita raíz → izquierda → derecha. Útil para copiar o serializar el árbol.	Exportación de árboles
Recorrido post-order	Visita izquierda → derecha → raíz. Útil para eliminar un árbol completo.	Liberación de memoria
Altura del árbol	Número de niveles desde la raíz hasta la hoja más profunda.	Análisis de desempeño
Nodo hoja	Nodo sin hijos. Representa el extremo de una rama del árbol.	ABB, árboles de expresión
Estabilidad	Mantener el orden de llegada cuando dos elementos tienen la misma prioridad.	Colas prioritarias
Timestamp	Marca de tiempo usada para registrar llegadas y calcular tiempos de espera.	Reportes y métricas
SLA	Objetivo de nivel de servicio. Por ejemplo, el 90% de atenciones en menos de 10 min.	Paneles de control

PROYECTO 1
Historial de navegación con lista doblemente enlazada

Estructura principal: Listas

Objetivo aplicado

Simular el historial de navegación de un navegador web, permitiendo avanzar y retroceder entre páginas visitadas.

Contexto

Un navegador almacena cada página visitada. El usuario puede retroceder (botón "Atrás") y avanzar (botón "Adelante") entre páginas. También puede borrar entradas del historial o limpiar todo.

Estructuras y algoritmos a usar

- Lista doblemente enlazada para almacenar el historial (cada nodo apunta al anterior y al siguiente).
- Puntero de posición actual que indica la página activa en el historial.

Justificación de la EDD

La lista doblemente enlazada es ideal porque permite moverse hacia adelante y hacia atrás con O(1) desde la posición actual. Un arreglo requeriría desplazar elementos al insertar o eliminar, lo cual es costoso. Una lista simple solo permitiría retroceder, no avanzar.

Requisitos funcionales mínimos

- Registrar visita a una URL (nodo nuevo al avanzar borra el historial futuro).
- Retroceder y avanzar una página a la vez.
- Eliminar una página específica del historial.
- Limpiar todo el historial.
- Mostrar el historial completo indicando la página actual.

Cómo explicar la aplicación de EDD

- Demostrar que insertar al visitar una nueva página es O(1).
- Explicar por qué al visitar una nueva página se descarta el historial futuro.
- Mostrar la navegación bidireccional usando los punteros anterior y siguiente.

Datos/insumos sugeridos

- Archivo de texto con secuencia de acciones: VISITAR, ATRAS, ADELANTE, ELIMINAR.
- Mínimo 50 acciones para una simulación representativa.

Pruebas mínimas

- Retroceder cuando ya se está en la primera página (caso borde).
- Avanzar cuando ya se está en la última página (caso borde).
- Visitar una nueva página desde un punto intermedio del historial.
- Eliminar la página actual y verificar que el puntero se ajusta correctamente.

Extensiones opcionales

- Guardar y cargar el historial desde un archivo CSV.
- Limitar el historial a un máximo de N páginas (eliminar la más antigua).

PROYECTO 2
Lista de reproducción de música con lista circular

Estructura principal: Listas

Objetivo aplicado

Gestionar una lista de reproducción de canciones que se reproduce de forma continua y cíclica, con soporte para modo aleatorio y repetición.

Contexto

Una aplicación de música permite al usuario crear listas de reproducción. El reproductor pasa automáticamente a la siguiente canción y al terminar la última vuelve a la primera. El usuario puede agregar, eliminar y reordenar canciones.

Estructuras y algoritmos a usar

- Lista circular doblemente enlazada para las canciones (el último enlaza con el primero).
- Puntero a la canción actualmente en reproducción.

Justificación de la EDD

La lista circular elimina la necesidad de verificar si se llegó al final: el puntero simplemente avanza al siguiente nodo, que siempre existe. La doble dirección permite ir a la canción anterior sin recorrer toda la lista. Un arreglo requeriría cálculo de módulo y sería más difícil de modificar dinámicamente.

Requisitos funcionales mínimos

- Agregar canción al final o en una posición específica.
- Eliminar canción por nombre o posición.
- Avanzar y retroceder canción.
- Mostrar la lista completa indicando la canción actual.
- Reporte: duración total de la lista y número de canciones.

Cómo explicar la aplicación de EDD

- Mostrar cómo la circularidad evita la verificación de fin de lista.
- Demostrar inserción y eliminación en O(1) dada la posición.
- Explicar el recorrido cíclico para el modo de reproducción continua.

Datos/insumos sugeridos

- CSV con canciones: título, artista, duración en segundos.
- Archivo de acciones: AGREGAR, ELIMINAR, SIGUIENTE, ANTERIOR.

Pruebas mínimas

- Lista con una sola canción: avanzar y retroceder siempre regresa a ella.
- Eliminar la canción actual y verificar que el reproductor pasa a la siguiente.
- Agregar una canción en la posición 1 de una lista con 20 canciones.

Extensiones opcionales

- Modo aleatorio usando un generador de posición pseudoaleatoria.
- Guardar la lista en un archivo y restaurarla al iniciar.

PROYECTO 3
Carrito de compras en línea con lista simplemente enlazada

Estructura principal: Listas

Objetivo aplicado

Modelar el carrito de compras de una tienda virtual, permitiendo agregar, eliminar y modificar productos antes de confirmar el pedido.

Contexto

Una tienda virtual permite a los usuarios agregar productos a su carrito, cambiar cantidades, eliminar artículos y calcular el total. El carrito debe mantener el orden en que se agregaron los productos.

Estructuras y algoritmos a usar

- Lista simplemente enlazada para los artículos del carrito (cada nodo es un producto con cantidad y precio).
- Lista simplemente enlazada separada para el historial de pedidos confirmados.

Justificación de la EDD

La lista simplemente enlazada es suficiente porque el recorrido es siempre de inicio a fin. No se necesita navegación inversa. Es más eficiente en memoria que la lista doble y más flexible que un arreglo para insertar y eliminar productos en cualquier posición.

Requisitos funcionales mínimos

- Agregar producto (nombre, precio unitario, cantidad).
- Eliminar producto por nombre.
- Modificar cantidad de un producto existente.
- Calcular el total del carrito con descuentos opcionales.
- Confirmar pedido: pasar el carrito al historial y vaciarlo.
- Mostrar historial de pedidos anteriores.

Cómo explicar la aplicación de EDD

- Justificar por qué la lista simple es suficiente para este caso.
- Demostrar inserción al inicio (O(1)) vs. inserción ordenada (O(n)).
- Mostrar cómo se recorre la lista para calcular el total.

Datos/insumos sugeridos

- CSV de productos disponibles: código, nombre, precio.
- Secuencia de acciones: AGREGAR, ELIMINAR, MODIFICAR, CONFIRMAR.

Pruebas mínimas

- Eliminar un producto que no existe en el carrito (manejo de error).
- Carrito vacío al confirmar (caso borde).
- Agregar el mismo producto dos veces: acumular cantidad o crear nodo nuevo (definir política).

Extensiones opcionales

- Aplicar cupones de descuento por monto o por porcentaje.
- Ordenar el carrito por precio o por nombre antes de mostrar.

PROYECTO 4
Gestor de tareas pendientes con lista doblemente enlazada

Estructura principal: Listas

Objetivo aplicado

Organizar tareas pendientes de un usuario con soporte para mover, priorizar y completar tareas en cualquier posición de la lista.

Contexto

Un gestor de tareas personal permite crear tareas con título, descripción y fecha límite. El usuario puede reordenarlas, marcarlas como completadas y archivarlas. Las tareas completadas pasan a una lista separada.

Estructuras y algoritmos a usar

- Lista doblemente enlazada para las tareas pendientes.
- Lista doblemente enlazada para las tareas completadas (archivo).
- Puntero a la tarea actualmente seleccionada.

Justificación de la EDD

La lista doble permite mover una tarea hacia arriba o hacia abajo en la lista sin necesidad de recorrerla desde el inicio. Esto es importante para la funcionalidad de reordenamiento. La lista simple no permitiría mover tareas hacia arriba de forma eficiente sin recorrer toda la lista.

Requisitos funcionales mínimos

- Agregar tarea al final o al inicio de la lista.
- Mover tarea hacia arriba o hacia abajo en la lista.
- Marcar tarea como completada (pasa a la lista de archivo).
- Eliminar tarea de la lista pendiente o del archivo.
- Buscar tarea por título.
- Mostrar todas las tareas pendientes y las completadas.

Cómo explicar la aplicación de EDD

- Demostrar cómo mover una tarea hacia arriba es O(1) con lista doble.
- Explicar la diferencia con una lista simple para la misma operación.
- Mostrar el recorrido hacia adelante para visualizar y hacia atrás para buscar desde el final.

Datos/insumos sugeridos

- CSV con tareas iniciales: título, descripción, fecha límite.
- Secuencia de acciones: AGREGAR, MOVER_ARRIBA, MOVER_ABAJO, COMPLETAR, ELIMINAR.

Pruebas mínimas

- Mover la primera tarea hacia arriba (caso borde).
- Mover la última tarea hacia abajo (caso borde).
- Completar todas las tareas y verificar que la lista pendiente queda vacía.

Extensiones opcionales

- Ordenar tareas por fecha límite.
- Exportar lista de tareas pendientes a un archivo de texto.

PROYECTO 5
Sistema de impresión con cola de trabajos

Estructura principal: Colas

Objetivo aplicado

Simular una cola de trabajos de impresión en una oficina, gestionando el orden de llegada y los recursos disponibles.

Contexto

Una oficina tiene una impresora compartida. Los usuarios envían documentos a imprimir y estos se atienden en orden de llegada. Algunos trabajos pueden ser marcados como urgentes y pasan al frente de la cola.

Estructuras y algoritmos a usar

- Cola FIFO (lista simplemente enlazada) para trabajos normales.
- Cola prioritaria para trabajos urgentes (se atienden antes que los normales).
- Lista simplemente enlazada para el registro histórico de trabajos completados.

Justificación de la EDD

La cola FIFO refleja exactamente el comportamiento justo de una impresora: primero en llegar, primero en imprimir. No se usa un arreglo porque insertar al inicio (para urgentes) sería costoso. La lista enlazada permite agregar al final e insertar al inicio con O(1).

Requisitos funcionales mínimos

- Enviar trabajo a imprimir (usuario, nombre de documento, número de páginas, tipo: normal o urgente).
- Procesar el siguiente trabajo (atender el primero de la cola).
- Cancelar un trabajo pendiente por nombre.
- Mostrar la cola actual con posición y tiempo estimado.
- Reporte: tiempo promedio de espera, trabajos completados por usuario.

Cómo explicar la aplicación de EDD

- Demostrar que encolar es O(1) y desencolar es O(1) con lista enlazada.
- Explicar cómo los urgentes se insertan al inicio sin afectar el orden de los normales.
- Mostrar por qué no se usa un arreglo: insertar al inicio sería O(n).

Datos/insumos sugeridos

- CSV de trabajos: usuario, documento, páginas, tipo, hora de envío.
- Velocidad de impresión configurable: páginas por minuto.

Pruebas mínimas

- Cola vacía al intentar imprimir (caso borde).
- Varios trabajos urgentes consecutivos: verificar orden entre ellos.
- Cancelar el único trabajo en la cola.

Extensiones opcionales

- Múltiples impresoras con asignación automática al menos ocupado.
- Alerta si un trabajo lleva más de X minutos esperando.

PROYECTO 6
Atención en banco con preferencia para adultos mayores

Estructura principal: Colas

Objetivo aplicado

Diseñar un sistema de atención justa en cajas con prioridad para personas mayores de 60 años, personas con discapacidad y embarazadas, sin descuidar la cola general.

Contexto

Un banco atiende clientes con tres categorías: Preferente (mayores de 60 años), Prioritaria (embarazadas o discapacidad) y General. Hay varias cajas y se aplica una regla de equidad para evitar esperas indefinidas.

Estructuras y algoritmos a usar

- Tres colas FIFO separadas (lista simplemente enlazada) por categoría: Preferente, Prioritaria, General.
- Contador de atenciones consecutivas para aplicar la regla de equidad (envejecimiento).

Justificación de la EDD

Separar en tres colas simplifica la aplicación de reglas de prioridad. Una sola cola requeriría reordenar elementos al cambiar prioridades. La lista enlazada permite insertar al final en O(1) y atender desde el frente en O(1), lo cual es ideal para colas de atención.

Requisitos funcionales mínimos

- Registrar llegada de cliente (DNI, edad, categoría, hora de llegada).
- Asignar cliente a caja según prioridad y disponibilidad.
- Aplicar regla de equidad: máximo 2 preferentes consecutivos antes de atender 1 general (si hay en espera).
- Reporte: tiempo medio de espera por categoría, utilización de cajas.

Cómo explicar la aplicación de EDD

- Justificar por qué tres colas separadas facilitan las reglas de prioridad.
- Explicar el mecanismo de envejecimiento y cómo garantiza que la cola general avance.
- Demostrar que la inserción en cola es O(1) con lista enlazada.

Datos/insumos sugeridos

- CSV con llegadas: DNI, edad, categoría, hora.
- Configuración: número de cajas y política de equidad (por ejemplo 2P:1G).

Pruebas mínimas

- Escenario saturado con muchos preferentes: verificar que la cola general también avanza.
- Escenario sin clientes preferentes: operación como cola normal.
- Caja fuera de servicio temporal: redistribución de atención.

Extensiones opcionales

- Asignación dinámica de cajas según demanda por franja horaria.
- Dashboard con métricas en tiempo real: tiempo promedio, máximo y p90.

PROYECTO 7
Atención en municipalidad con múltiples servicios y cola de apoyo

Estructura principal: Colas

Objetivo aplicado

Balancear las filas de atención por tipo de trámite (licencias, pagos, informes) activando colas de apoyo cuando la demanda supera un umbral.

Contexto

La municipalidad tiene ventanillas por servicio y la demanda varía a lo largo del día. Cuando una cola crece demasiado, se abre una ventanilla adicional para ese servicio.

Estructuras y algoritmos a usar

- Múltiples colas FIFO (listas enlazadas) por tipo de servicio.
- Cola de apoyo: se activa cuando la cola principal supera el umbral. Los turnos se intercalan (uno de la principal, uno de la de apoyo).
- Lista circular para calcular estadísticas en ventanas de tiempo (últimos N minutos).

Justificación de la EDD

Tener colas separadas por servicio evita que un trámite lento bloquee a quienes necesitan otro servicio. La lista circular para estadísticas permite descartar datos antiguos automáticamente al avanzar el puntero, sin necesidad de eliminar nodos.

Requisitos funcionales mínimos

- Registrar llegada con tipo de trámite.
- Asignar a ventanilla o cola de apoyo si hay saturación.
- Dashboard de métricas: tiempo promedio, mediana, máximo y percentil 90 (p90).
- Cerrar la cola de apoyo cuando la demanda se normaliza.

Cómo explicar la aplicación de EDD

- Defender el uso de colas separadas por servicio frente a una bolsa única.
- Explicar cómo la lista circular permite calcular métricas en ventanas de tiempo sin acumular todos los datos.
- Justificar el intercalado de turnos entre la cola principal y la de apoyo.

Datos/insumos sugeridos

- CSV con llegadas por franja horaria: usuario, tipo de trámite, hora.
- Umbral configurable para activar la cola de apoyo.

Pruebas mínimas

- Saturación en un solo servicio y activación correcta de la cola de apoyo.
- Cambio abrupto de demanda (llegada masiva de usuarios).
- Cierre de cola de apoyo cuando el flujo se normaliza.

Extensiones opcionales

- Prioridades por tipo de trámite con reglas de equidad.
- Alerta cuando el tiempo de espera supera el SLA definido.

PROYECTO 8
Centro de llamadas multicanal con lista circular de agentes

Estructura principal: Colas

Objetivo aplicado

Asignar tickets de distintos canales (teléfono, chat, correo) a agentes de forma equilibrada usando round-robin, manejando pausas y retornos del personal.

Contexto

El centro recibe consultas por tres canales. Cada canal tiene su propia cola. Los agentes se turnan para atender y pueden entrar en pausa sin perder su posición en la rotación.

Estructuras y algoritmos a usar

- Tres colas FIFO separadas por canal (teléfono, chat, correo).
- Lista circular de agentes activos para repartir turnos (round-robin).
- Lista doblemente enlazada para pausar y reinsertar agentes sin perder su posición relativa.

Justificación de la EDD

La lista circular es perfecta para round-robin: al llegar al último agente simplemente se vuelve al primero, sin necesidad de calcular posiciones. La lista doble para los agentes pausados permite extraer y reinsertar en O(1) dado el nodo, sin recorrer toda la lista.

Requisitos funcionales mínimos

- Registrar llegada de ticket con canal, cliente y hora.
- Asignar al siguiente agente disponible según round-robin.
- Pausar y reactivar agentes manteniendo la rotación.
- Métricas: tiempo de espera por canal y carga por agente.

Cómo explicar la aplicación de EDD

- Demostrar cómo la lista circular garantiza una distribución equitativa.

- Explicar el uso de la lista doble para extraer y devolver agentes a la rotación.
- Mostrar cómo las colas separadas por canal mantienen el orden de llegada de cada tipo.

Datos/insumos sugeridos

- CSV de llegadas por canal: canal, cliente, hora.
- Lista de agentes con horarios de pausa simulados.

Pruebas mínimas

- Saturación de un canal con asignación equilibrada entre agentes.
- Agente en pausa y retorno: la rotación se mantiene correctamente.
- Todos los agentes en pausa simultáneamente (caso borde).

Extensiones opcionales

- Preferencia por canal (por ejemplo, teléfono antes que correo).
- Envejecimiento para tickets de correo con mucho tiempo de espera.

PROYECTO 9
Estacionamiento en una sola vía con carril auxiliar

Estructura principal: Pilas

Objetivo aplicado

Gestionar entradas y salidas en un estacionamiento lineal usando una pila auxiliar para extraer autos intermedios sin bloquear el flujo.

Contexto

Un garaje en línea tiene entre 10 y 50 plazas. Para sacar un auto que no está al final, se deben mover temporalmente los autos que están delante.

Estructuras y algoritmos a usar

- Pila principal para los autos estacionados (LIFO).
- Pila auxiliar para mover y reinsertar autos cuando sale uno intermedio.
- Lista simplemente enlazada para el historial de movimientos (entradas y salidas).

Justificación de la EDD

El garaje en línea es físicamente una pila: el último en entrar es el primero en salir. Para sacar un auto del fondo se necesita una segunda pila auxiliar que actúa como área de espera temporal. Esta combinación permite resolver el problema sin modificar la estructura física del estacionamiento.

Requisitos funcionales mínimos

- Registrar entrada de auto (placa, hora) y validar capacidad.
- Procesar salida por placa: mover autos a la pila auxiliar, extraer el buscado, reponer los autos.
- Calcular tiempo de estancia y costo al salir.
- Reportes: ocupación, tiempo promedio de estancia, movimientos por salida.

Cómo explicar la aplicación de EDD

- Justificar por qué el flujo físico del garaje se modela como pila (LIFO).
- Explicar el uso de la pila auxiliar para salidas intermedias.
- Analizar el impacto: salidas del último puesto son O(1), intermedias son O(n).

Datos/insumos sugeridos

- CSV de eventos: tipo (entrada/salida), placa, hora.
- Capacidad del garaje y tarifa por hora configurables.

Pruebas mínimas

- Salida del último auto en entrar (caso simple, O(1)).
- Salida del primer auto en entrar (requiere mover todos los demás).
- Garaje lleno ante nueva entrada (rechazo).
- Salida de placa inexistente (manejo de error).

Extensiones opcionales

- Reservas de plazas con lista de autorizados.
- Tarifa diferenciada por franja horaria.

PROYECTO 10
Verificador de expresiones con paréntesis y corchetes balanceados

Estructura principal: Pilas

Objetivo aplicado

Verificar si las expresiones matemáticas o de código fuente tienen sus símbolos de agrupación correctamente balanceados y anidados.

Contexto

Un compilador o calculadora necesita verificar que cada paréntesis, corchete y llave que abre tenga su correspondiente cierre en el orden correcto. Por ejemplo: {[()]} es válido, pero {[(])} no lo es.

Estructuras y algoritmos a usar

- Pila para rastrear los símbolos de apertura encontrados.
- Lista simplemente enlazada para almacenar las expresiones a verificar y sus resultados.

Justificación de la EDD

La pila es la estructura natural para este problema porque el último símbolo de apertura siempre debe cerrarse primero (LIFO). Al encontrar un símbolo de cierre, se verifica contra el tope de la pila. Ninguna otra estructura lineal permite hacer esto de manera tan directa.

Requisitos funcionales mínimos

- Leer una expresión y verificar si está balanceada.
- Identificar la posición exacta donde se detecta el error.
- Procesar múltiples expresiones desde un archivo.
- Reporte: número de expresiones válidas e inválidas con detalle de errores.

Cómo explicar la aplicación de EDD

- Demostrar paso a paso cómo la pila procesa una expresión.
- Explicar los tres casos: símbolo de cierre sin apertura, apertura sin cierre, y cierre incorrecto.
- Mostrar que cada carácter se procesa en O(1) y la expresión completa en O(n).

Datos/insumos sugeridos

- Archivo de texto con al menos 50 expresiones entre válidas e inválidas.
- Incluir casos con paréntesis, corchetes y llaves combinados.

Pruebas mínimas

- Expresión vacía (válida por definición).
- Expresión con solo aperturas sin cierres.
- Expresión con solo cierres sin aperturas.
- Expresiones con anidamiento profundo correcto e incorrecto.

Extensiones opcionales

- Extender para verificar etiquetas HTML (apertura y cierre de tags).
- Sugerir la corrección automática de la expresión inválida.

PROYECTO 11
Control de seguridad en aeropuerto: bandejas y filas

Estructura principal: Pilas

Objetivo aplicado

Gestionar las filas de pasajeros y el uso de bandejas en los controles de seguridad de un aeropuerto, optimizando el flujo y registrando incidencias.

Contexto

En seguridad, los pasajeros forman filas y usan bandejas para sus pertenencias. Las bandejas se apilan al ser devueltas y se toman del tope de la pila. Cada línea tiene su propia pila de bandejas y cola de pasajeros.

Estructuras y algoritmos a usar

- Cola FIFO (lista simplemente enlazada) de pasajeros por línea de control.
- Pila de bandejas por línea (LIFO): se toma la bandeja del tope y se devuelve al tope.
- Lista simplemente enlazada para registrar incidencias (objetos prohibidos, revisiones adicionales).

Justificación de la EDD

Las bandejas apiladas siguen exactamente el comportamiento LIFO: siempre se toma la que está encima. Las colas modelan correctamente la espera ordenada de pasajeros. La separación de estructuras por línea permite escalar el sistema añadiendo más líneas de control.

Requisitos funcionales mínimos

- Registrar llegada de pasajero y asignarlo a la línea con menor cola.
- Entregar bandeja desde la pila y devolverla al finalizar el control.
- Marcar incidencias y generar reportes por hora.
- Métricas: tiempo de espera por línea, uso de bandejas, incidencias por tipo.

Cómo explicar la aplicación de EDD

- Justificar por qué las bandejas se modelan como pila.
- Demostrar que entregar y recibir bandejas es O(1) con pila.
- Explicar cómo la lista enlazada facilita el registro de incidencias sin un tamaño fijo.

Datos/insumos sugeridos

- CSV de llegadas por hora y número de líneas disponibles.
- Número de bandejas disponibles por línea configurable.

Pruebas mínimas

- Pico de llegadas con bandejas limitadas: pasajero sin bandeja disponible.
- Desbalance entre líneas: una línea mucho más rápida que otra.
- Registro de múltiples incidencias en la misma línea.

Extensiones opcionales

- Balanceo: mover pasajeros a otra línea si se supera un umbral de espera.
- Estadísticas por tipo de incidencia al final del turno.

PROYECTO 12
Biblioteca universitaria: índice de libros por ABB

Estructura principal: Arboles Binarios

Objetivo aplicado

Construir un índice de libros que permita insertar, buscar y eliminar registros de forma eficiente usando un Árbol Binario de Búsqueda.

Contexto

La biblioteca universitaria necesita un sistema de búsqueda rápida por ISBN y por título. El índice debe mantenerse ordenado en todo momento y permitir listar los libros en orden alfabético.

Estructuras y algoritmos a usar

- Árbol Binario de Búsqueda (ABB) como índice principal (clave: ISBN).
- Segundo ABB para búsqueda por título.
- Recorridos in-order para listar en orden, pre-order para exportar, y por niveles para visualizar.

Justificación de la EDD

El ABB permite buscar, insertar y eliminar en O(log n) en el caso promedio, mucho más eficiente que recorrer una lista enlazada en O(n). El recorrido in-order produce los libros ordenados por ISBN automáticamente sin necesidad de un algoritmo de ordenamiento adicional.

Requisitos funcionales mínimos

- Cargar libros desde un archivo CSV (ISBN, título, autor, año).
- Insertar libro evitando duplicados de ISBN.
- Buscar por ISBN o por título.
- Eliminar libro manejando los tres casos: nodo hoja, un hijo, dos hijos.
- Listar todos los libros en orden por ISBN (recorrido in-order).
- Mostrar estadísticas del árbol: altura, número de nodos, nodos hoja.

Cómo explicar la aplicación de EDD

- Explicar cómo se mantiene el invariante del ABB en cada inserción.
- Demostrar los tres casos de eliminación con ejemplos.
- Discutir cómo el orden de inserción afecta la altura y el desempeño del árbol.

Datos/insumos sugeridos

- CSV de libros con entre 1 000 y 10 000 registros.
- Incluir tanto inserciones en orden aleatorio como en orden ascendente para comparar alturas.

Pruebas mínimas

- Insertar libros en orden ascendente de ISBN: árbol degenerado (lista).
- Insertar en orden aleatorio y comparar altura con el caso anterior.
- Eliminar el nodo raíz y verificar que el árbol se reestructura correctamente.

Extensiones opcionales

- Exportar el árbol en formato DOT para visualización con Graphviz.
- Implementar búsqueda por rango de años de publicación.

PROYECTO 13
Sistema de decisiones médicas con árbol binario de expresiones

Estructura principal: Arboles Binarios

Objetivo aplicado

Representar y evaluar un árbol de decisiones clínicas para orientar al médico en el diagnóstico diferencial de síntomas comunes.

Contexto

Un sistema de apoyo al diagnóstico guía al médico con preguntas de sí/no sobre los síntomas del paciente. Según las respuestas, el árbol navega hacia el diagnóstico más probable. El árbol puede actualizarse agregando nuevas ramas.

Estructuras y algoritmos a usar

- Árbol binario de decisión: cada nodo interno es una pregunta, cada hoja es un diagnóstico.
- Rama izquierda: respuesta NO. Rama derecha: respuesta SÍ.
- Lista simplemente enlazada para registrar el camino de decisiones tomadas en cada consulta.

Justificación de la EDD

El árbol binario es la estructura natural para modelar decisiones de sí/no porque cada nodo tiene exactamente dos hijos posibles. El recorrido pre-order permite reconstruir el árbol completo para guardarlo o cargarlo. Una lista enlazada no permite esta jerarquía de decisiones.

Requisitos funcionales mínimos

- Cargar el árbol de decisiones desde un archivo (formato: pregunta, respuesta_si, respuesta_no).
- Recorrer el árbol interactivamente respondiendo preguntas.
- Registrar el camino tomado para cada consulta.

- Agregar una nueva rama (nueva pregunta y diagnósticos) en un nodo hoja.
- Exportar el árbol completo usando recorrido pre-order.

Cómo explicar la aplicación de EDD

- Demostrar el recorrido del árbol paso a paso con un caso de ejemplo.
- Explicar por qué el recorrido pre-order es adecuado para exportar y reconstruir el árbol.
- Justificar el árbol binario frente a una lista de reglas secuenciales.

Datos/insumos sugeridos

- Archivo de texto con el árbol de decisiones para al menos 5 diagnósticos y 10 preguntas.
- Casos de prueba: conjunto de respuestas que llevan a cada diagnóstico.

Pruebas mínimas

- Recorrer el árbol hasta un nodo hoja (diagnóstico encontrado).
- Intentar agregar una rama en un nodo que no es hoja (manejo de error).
- Exportar el árbol y cargarlo nuevamente: verificar que es idéntico.

Extensiones opcionales

- Árbol con más de dos ramas por nodo (árbol n-ario) para respuestas múltiples.
- Estadísticas de uso: diagnósticos más frecuentes.

PROYECTO 14
Calculadora de expresiones aritméticas con árbol de expresión

Estructura principal: Arboles Binarios

Objetivo aplicado

Construir un árbol de expresión a partir de una expresión aritmética en notación infija, evaluarla y convertirla a notación prefija y postfija.

Contexto

Los compiladores e intérpretes matemáticos convierten expresiones como (3+5)*2 en árboles de expresión para evaluarlas o transformarlas. Este proyecto simula ese proceso de forma simplificada.

Estructuras y algoritmos a usar

- Árbol binario de expresión: hojas son operandos (números), nodos internos son operadores (+, -, *, /).
- Pila auxiliar para la construcción del árbol a partir de la expresión en notación postfija.
- Lista simplemente enlazada para almacenar las expresiones a procesar y sus resultados.

Justificación de la EDD

El árbol de expresión separa la estructura de la expresión de su evaluación. El recorrido in-order da la notación infija, el pre-order da la prefija (notación polaca) y el post-order da la postfija (notación polaca inversa). Esta versatilidad no es posible con ninguna estructura lineal.

Requisitos funcionales mínimos

- Leer una expresión en notación postfija y construir el árbol de expresión.
- Evaluar el árbol (recorrido post-order) y mostrar el resultado.
- Generar la notación infija (con paréntesis correctos) desde el árbol.

- Generar la notación prefija desde el árbol.
- Procesar múltiples expresiones desde un archivo y reportar resultados.

Cómo explicar la aplicación de EDD

- Mostrar paso a paso cómo la pila construye el árbol desde la notación postfija.
- Demostrar los tres recorridos (in-order, pre-order, post-order) con el mismo árbol.
- Explicar por qué el recorrido post-order evalúa correctamente la expresión.

Datos/insumos sugeridos

- Archivo con al menos 20 expresiones en notación postfija.
- Incluir operaciones anidadas y con números de varios dígitos.

Pruebas mínimas

- Expresión con un solo número (árbol de un solo nodo).
- División por cero: manejo del error sin colapsar el árbol.
- Expresión con operaciones de distintos niveles de precedencia.
- Expresión postfija inválida (manejo de error).

Extensiones opcionales

- Soporte para operador de potencia y paréntesis en la entrada.
- Simplificación de expresiones: detectar 0*x = 0, 1*x = x, etc.